KubeCon | CloudNativeCon
Europe 2026

ArgoCon
EUROPE

Empowering Autonomy:BYD’s Journey Taming Million-Task Scale with Argo Workflows

Bao Zhang,BYD
Shuangkun Tian, Alibaba Cloud

About US

Autonomy Engineer at BYD

Technical Expert at Alibaba Cloud

Agenda

KubeCon CloudNativeCon Europe 2026

- **Modernizing Autonomous Driving Workflows**
  1) **The Challenge:** Automatic Annotation - System Complexity & Engineering Difficulties
  2) **The Pain:** Airflow Limitations - Why We Moved Away
  3) **The Solution:** Argo Workflows - Embracing Cloud-Native Orchestration
  4) **The Architecture:** Multi-Cluster Deployment for Scalability
  5) **The Results:** Key Achievements with Argo Workflows
  6) **Beyond Argo:** RayCluster Integration & Complementary Technologies
- **Large-scale workflow optimizations**
- **Key Takeaways**

01 The Challenge

Automatic Annotation:
System Complexity & Engineering Difficulties

System Complexity of Automatic Annotation

- **Massive Computational Requirements:** Processing vast amounts of sensor data (LiDAR, camera, radar) demands enormous GPU and CPU resources.
- **Complex Data Pipelines:** The workflow involves multiple stages—data ingestion, preprocessing, model inference, and post-processing—each with its own dependencies.
- **Dynamic Resource Management:** Efficiently managing heterogeneous hardware (different GPU models, CPU instances) and scaling resources dynamically is critical.
- **High Throughput & Low Latency:** To keep up with test fleet data generation, the system must process tasks with high throughput and minimal delay.
- **Fault Tolerance & Reliability:** Ensuring the pipeline is resilient to failures and can recover gracefully without losing progress is essential.

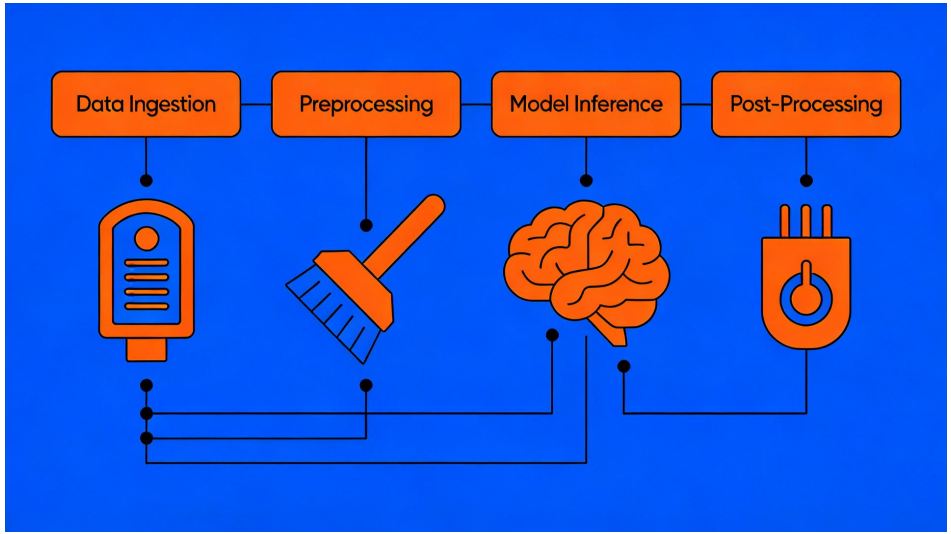


Figure: Autonomous Driving Data Processing Pipeline

02 The Pain

Airflow Limitations: Why We Moved Away

Limitations of Airflow at Scale

1. Scalability Bottleneck

The traditional Scheduler/Worker model struggled with the sheer volume of tasks, leading to frequent state synchronization issues and performance degradation at scale.

2. Mutable DAGs

A critical flaw was that modifying a DAG definition would affect already running pipelines, leading to unpredictable behavior and operational instability.

3. Non-GitOps

Airflow's lack of native GitOps support made it difficult to manage, version, and audit our workflows as code, which is essential for a modern DevOps practice.

bottleneck

mutability

non-GitOps

03 The Solution

Argo Workflows:
Embracing Cloud-Native Orchestration

Multi-Cluster Argo Architecture

- **Multiple K8s Clusters**

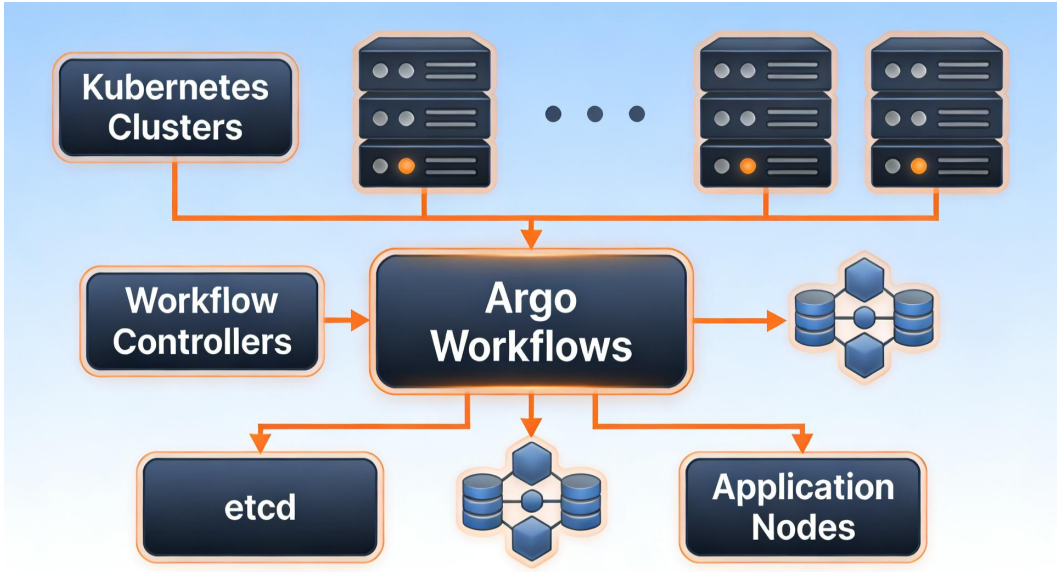
Deployed Argo across clusters to isolate fault domains and distribute load, reducing pressure on single cluster control planes.

- **Argo Workflow Controller per Cluster**

Each cluster runs its own controller, ensuring high availability and performance.

- **Centralized Management & Monitoring**

Unified view for management despite distributed setup, ensuring operational simplicity.



Resource Management & Scheduling

Hardware Resources

- **GPU:** Leveraging Alibaba's self-developed GPU chips for high-performance AI workloads.
- **CPU:** Utilizing Alibaba Cloud ECS instances combined with elastic machines for flexible and cost-effective general computing.

Scheduling Strategy

- **Argo Workflows:** Orchestrates the entire pipeline.
- **ElasticQuotaTree:** An Alibaba Cloud-developed tool for fine-grained quota allocation and scheduling management, ensuring efficient resource utilization across different teams and projects.

04 The Results

Key Metrics & Improvements with Argo

Key Achievements with Argo Workflows

- **Unprecedented Scalability**
  - Successfully supporting millions of concurrent tasks, a leap from Airflow's limitations.
- **Enhanced Stability**
  - Dramatically improved reliability and fault tolerance, with a near 99.9% success rate for workflows.
- **Operational Efficiency**
  - Pipeline execution times were drastically reduced, some by as much as 11x, leading to faster iteration cycles.
- **GitOps Compliance**
  - Full adoption of GitOps practices for workflow management, enabling better version control, auditability, and collaboration.
- **Cost Optimization**
  - More efficient resource utilization led to a reduction in overall compute costs by approximately 30%.

05 Beyond Argo

RayCluster Integration & Complementary Technologies

Argo Workflows vs. RayCluster

KubeCon CloudNativeCon Europe 2026

Feature	Argo Workflows	RayCluster
Scheduling	Container-level	Process-level
Isolation	Excellent (per-pod)	Less strict (shared workers)
Resource Utilization	Good (with pod scheduling overhead)	Higher (shared resources)
Cluster Scale	Very large (thousands of pods)	<1000 pods
Priority Flexibility	Highly flexible	Less flexible
Best For	General-purpose orchestration, complex DAGs	High-throughput, parallel data processing

Managing RayCluster with Argo Workflows

KubeCon CloudNativeCon Europe 2026

Why not RayJob:

We found RayJob's interface to be less stable and flexible for our needs.

Argo as a Supervisor:

Instead, we use Argo Workflows to manage the entire lifecycle of RayClusters. Argo handles the provisioning, scaling, and termination of RayClusters as needed by our workflows.

Benefits:

This approach combines Argo's robust orchestration capabilities with Ray's powerful distributed computing framework, giving us the best of both worlds for specific workloads.

KubeCon | CloudNativeCon
Europe 2026

Optimization at Scale

Concurrency Control、Stability at Extreme Scale、Observability

Multi-Dimensional Concurrency Control

KubeCon CloudNativeCon Europe 2026

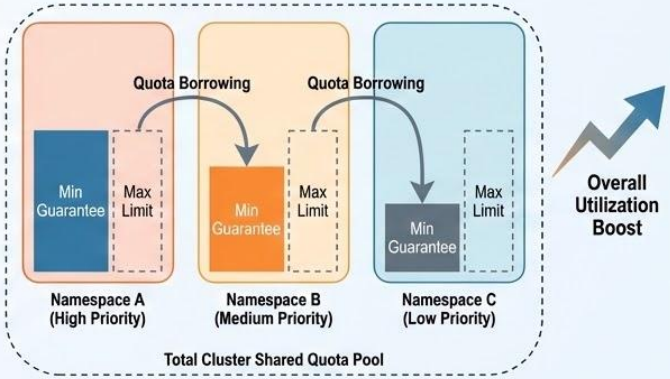
1. Utilizing Argo Workflows Native Capabilities

- **Task-Level Control**: Use `parallelism` to limit task concurrency within a single Workflow.
- **Resource-Level Synchronization**: Implement `Semaphore` (synchronization lock) for precise concurrency control across multiple Workflows or within a single Workflow, especially for GPU resources.
- **Goal**: Prevent excessive Pod creation leading to Pending states due to resource starvation, reducing pressure on the scheduler.
- **Multi-Team Isolation**: Configure different workflow concurrency limits for distinct Namespaces (e.g., Simulation Team, Data Processing Team).

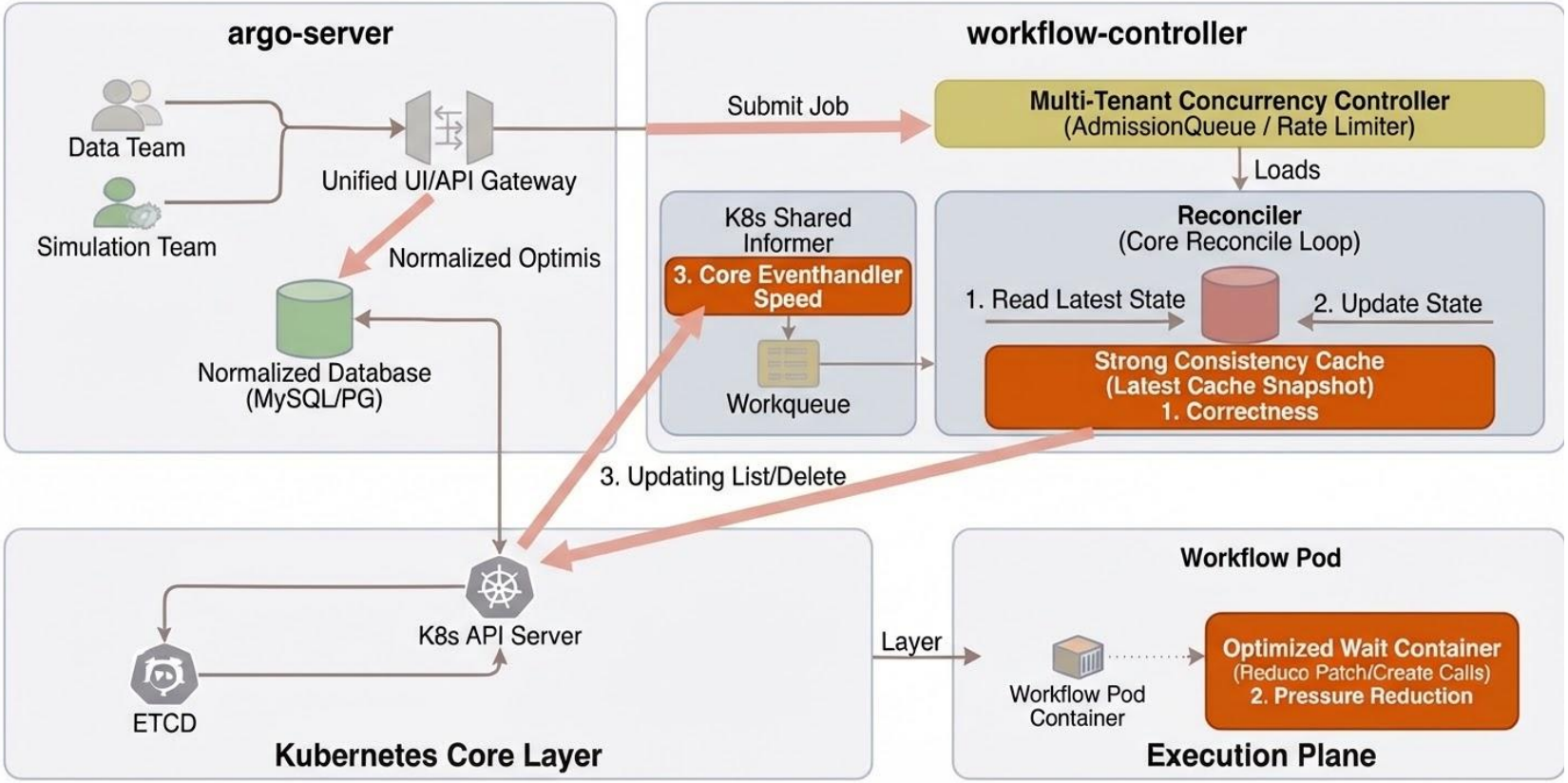
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: gpu-workflow-
spec:
  entrypoint: gpu-task-dag
  parallelism: 5  # Limit tasks within the workflow
  templates:
  - name: gpu-task-dag
    dag:
      tasks:
      - name: train-model
        template: train-model-template
  - name: train-model-template
    synchronization:
      semaphore:
        configMapKeyRef:
          name: gpu-semaphore
          key: workflow-concurrency
    container:
      image: gpu-image:latest
      resources:
        limits:
          nvidia.com/gpu: 1
```

2. Leveraging K8s Scheduler Quota Capabilities

- **Shared & Guaranteed Quota**: Multiple Namespaces share the overall cluster Quota with different priorities.
- **Elastic Borrowing Mechanism**: Configure different `Min` (guaranteed) and `Max` (limit) for Namespaces with varying priorities. Quota can be borrowed between Namespaces.
- **Effect**: Ensures basic resource guarantees for low-priority tasks during peaks, while allowing high-priority tasks to preempt resources based on minimum guarantees, significantly improving overall utilization.



Stability at Extreme Scale



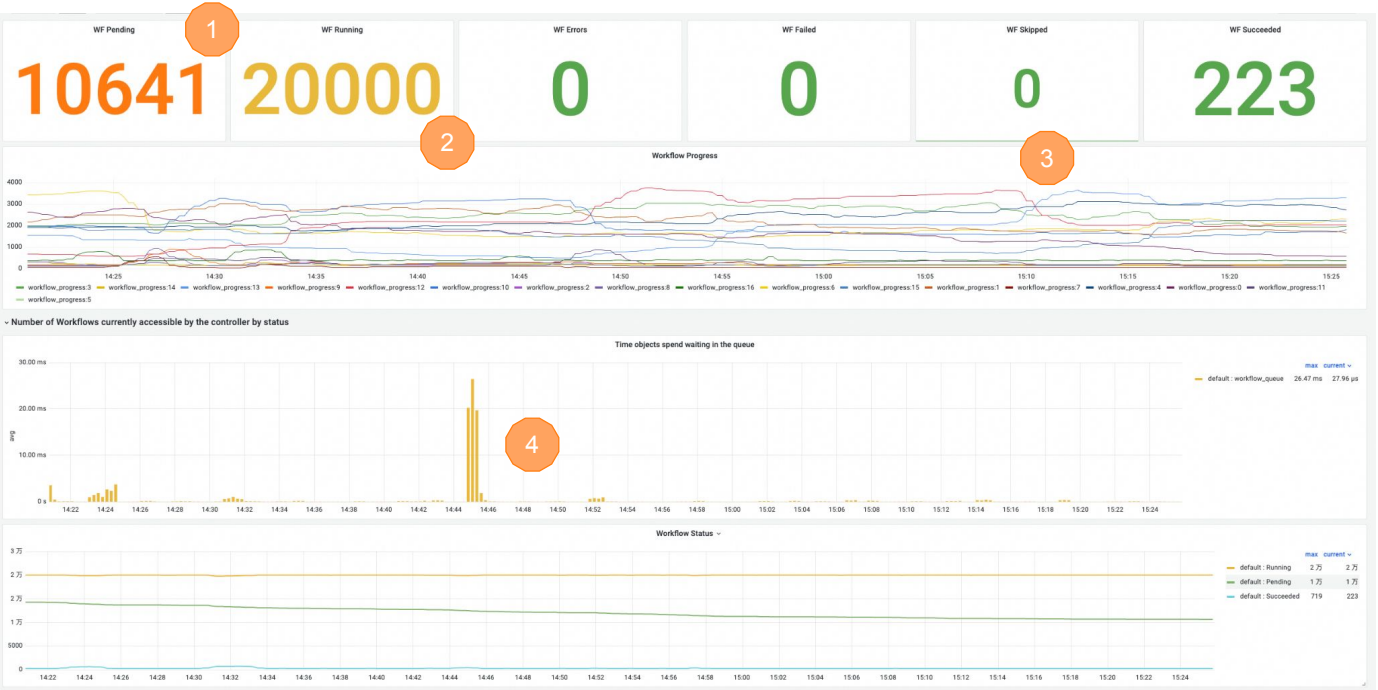
Observability and Performance

1) **Pending Queue Capacity:** From 10k to 200k+

2) **Concurrency:** > 20k+

3) **Observability:** Step-by-step progress visibility.

4) **Latency:** <50ms



Giving Back: Community Contributions

KubeCon CloudNativeCon Europe 2026

Category	Contribution (PR)
Stability & Fault Tolerance	Implemented retry logic for extreme scenarios (e.g., "Too Many Requests" errors).
	Ensured database access timeouts return immediately to prevent system stalling.
Efficiency & Performance	Reduced controller load by preventing duplicate creation of `WorkflowsTaskResults`
	Optimized batch cleanup of `WorkflowTaskResult`
	Enhanced performance through improved concurrency control and offloading tasks to user queues.
	Improved controller processing speed via caching `Workflow-controller` updates.
Extreme Scale Management	Ensured the status is set to Pending after large-scale retries.

Conclusion

Key Takeaways

- **Architecture Evolution**: Transitioned from Airflow to a **multi-cluster Argo Workflows** architecture to eliminate scalability bottlenecks and support million-task scale.
- **Infrastructure Integration**: Leveraged **Argo as a supervisor** to manage RayCluster life cycles, combining robust orchestration with high-throughput parallel data processing.
- **Performance at Scale**: Achieved extreme performance with **20k+ concurrent tasks**, 200k+ queue capacity, and a low **50ms scheduling latency**.
- **Efficiency & Impact**: Reduced execution times by up to **11x**, leading to a 30% reduction in overall compute costs.

KubeCon | CloudNativeCon

Europe 2026

Thank You !